Report
MPC Project

Rocket landing controller

École Polytechnique Fédérale de Lausanne
ME-425

Authors	**SCIPERs**
Killian Baillifard	393835
Alex Gochely	361135

Lausanne, January 11, 2026

Contents

1 Introduction

1.1 Goal

The goal of this project is to put in practice all the material seen in the MPC course. For this, we will be developing a MPC controller to **land a rocket autonomously** Space-X style.

1.2 State

The state space representation of the rocket uses 12 dimensions :

$$\begin{aligned} \boldsymbol{x} &= \begin{bmatrix} \boldsymbol{\omega}^T & \boldsymbol{\varphi}^T & \boldsymbol{v}^T & \boldsymbol{p}^T \end{bmatrix}^T \\ \boldsymbol{\omega} &= \begin{bmatrix} \omega_x & \omega_y & \omega_z \end{bmatrix}^T \\ \boldsymbol{\varphi} &= \begin{bmatrix} \varphi_x & \varphi_y & \varphi_z \end{bmatrix}^T \\ \boldsymbol{v} &= \begin{bmatrix} v_x & v_y & v_z \end{bmatrix}^T \\ \boldsymbol{p} &= \begin{bmatrix} p_x & p_y & p_z \end{bmatrix}^T \end{aligned} \tag{1.1}$$

1.3 Input

And the input vector with its constraints is :

$$\begin{aligned} &\boldsymbol{u} = \begin{bmatrix} \delta_1 & \delta_2 & P_{\text{avg}} & P_{\text{diff}} \end{bmatrix}^T \\ &|\delta_1|, |\delta_2| \leq 15^\circ \\ &|P_{\text{diff}}| \leq 20\% \\ &P_{\text{avg}} \in [10, 90]\% \end{aligned} \tag{1.2}$$

1.4 Dynamics

Most of the rocket dynamics are represented by nonlinear matrices. For example, the attitude of the rocket is $\boldsymbol{r} = \begin{bmatrix} \alpha & \beta & \gamma \end{bmatrix}$ with its rate of change given by :

$$\dot{\boldsymbol{r}} = \frac{1}{\cos(\beta)} \begin{bmatrix} \cos(\gamma) & -\sin(\gamma) & 0 \\ \sin(\gamma) \cdot \cos(\beta) & \cos(\gamma) \cdot \cos(\beta) & 0 \\ -\cos(\gamma) \cdot \sin(\beta) & \sin(\gamma) \cdot \sin(\beta) & \cos(\beta) \end{bmatrix} \boldsymbol{\omega} \tag{1.3}$$

The full dynamics of the rocket is expressed as :

$$\dot{\boldsymbol{x}} = f(\boldsymbol{x}, \boldsymbol{u}) \tag{1.4}$$

2 Linearization

2.1 Explanation

Linear model

Linearizing the system dynamics around the **equilibrium point** (i.e. hover at the origin), we get the following relationships inside of A :

$$\dot{\boldsymbol{r}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \boldsymbol{\omega} \qquad \dot{\boldsymbol{v}} = \begin{bmatrix} 0 & g & 0 \\ -g & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \boldsymbol{r} \qquad \dot{\boldsymbol{p}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \boldsymbol{v} \tag{2.1}$$

We can clearly see that all axes are separable. Likewise for B :

$$\dot{\boldsymbol{\omega}} = \begin{bmatrix} -65.47 & 0 & 0 \\ 0 & -65.47 & 0 \\ 0 & 0 & 0.10 \end{bmatrix} \begin{bmatrix} \delta_1 \\ \delta_2 \\ P_{\text{diff}} \end{bmatrix} \qquad \dot{\boldsymbol{v}} = \begin{bmatrix} 0 & -g & 0 \\ g & 0 & 0 \\ 0 & 0 & 0.15 \end{bmatrix} \begin{bmatrix} \delta_1 \\ \delta_2 \\ P_{\text{avg}} \end{bmatrix} \tag{2.2}$$

Linear and angular acceleration each depend on a different input.

Derivation

In this model, all nonlinear behaviors arise from trigonometric relationships. Taking equations 1.3, and evaluating it at the origin, we get :

$$\dot{\boldsymbol{r}} = \frac{1}{\cos(0)} \begin{bmatrix} \cos(0) & -\sin(0) & 0 \\ \sin(0)\cdot\cos(0) & \cos(0)\cdot\cos(0) & 0 \\ -\cos(0)\cdot\sin(0) & \sin(0)\cdot\sin(0) & \cos(0) \end{bmatrix} \boldsymbol{\omega} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \boldsymbol{\omega} = I\boldsymbol{\omega} \tag{2.3}$$

The same logic applies to all other equations, with the **equilibrium point** and the **small angle approximation** making every matrix a combination of **diagonal** and **anti-diagonal** elements.

Intuition

At **equilibrium** with **small angles**, one can look at the rocket like a **marble rolling on a tilting table**. Small tilts in x and y (δ_1 and δ_2) will mostly change velocity in the x and y directions.

Using the same analogy, **moving** the table **up or down** will not change x and y velocities much.

The only thing missing from this example is the **roll axis**, which is irrelevant for a marble, but much more **important for our rocket**, as it defines the difference between the **body** and **world** x and y directions.

That is why dynamics can be considered **independent** for x, y, z and roll axes (as long as the rocket body is aligned with the world coordinates).

3 Controller

Because of the small angle approximation and empirical experiments on descent speed, two additional constraints are added :

$$\begin{aligned} |\alpha| &\leq \frac{\pi}{18} = 10^\circ \\ |\beta| &\leq \frac{\pi}{18} = 10^\circ \\ 40\% &\leq P_{\text{avg}} \leq 80\% \end{aligned} \tag{3.1}$$

3.1 Velocity stabilization

3.1.1 Design procedure

Base controller

To work around the linearization point, all 4 controllers use the **delta formulation** :

$$\begin{aligned} \Delta x_k &= x_k - x_s \\ \Delta u_k &= u_k - u_s \\ \Delta x_0 &= x_0 - x_s \\ \Delta x_{k+1} &= A\Delta x_k + B\Delta u_k \end{aligned} \tag{3.2}$$

Delta variables are embedded into the problem using **equality constraints**. This allows to define the problem constraints with the base variables x and u.

x and y subsystems

Both x and y velocity controllers must respect state and input constraints :

$$\begin{aligned} |\alpha| &\leq 10^\circ \\ |\beta| &\leq 10^\circ \\ |\delta_1| &\leq 15^\circ \\ |\delta_2| &\leq 15^\circ \end{aligned} \tag{3.3}$$

To ensure **stability** and **recursive feasibility**, an **LQR** controller is used to compute a **terminal set** $\mathcal{O}_\infty$:

$$\begin{aligned} x_k &\in \mathcal{X} \quad \forall \quad k \in [0, N-1] \\ u_k &\in \mathcal{U} \quad \forall \quad k \in [0, N-1] \\ x_N &\in \mathcal{O}_\infty \end{aligned} \tag{3.4}$$

And the final cost is defined using the **terminal cost** Q_f of the terminal **LQR** controller:

$$\min \sum_{k=0}^{N-1} x_k^T Q x_k + u_k R u_k + x_N Q_f x_N \tag{3.5}$$

z and roll subsystems

The z and roll subsystems only have input constraints :

$$\begin{aligned} 40\% \leq & P_{\text{avg}} \leq 80\% \quad \forall \quad k \in [0, N-1] \\ & |P_{\text{diff}}| \leq 20\% \quad \forall \quad k \in [0, N-1] \end{aligned} \tag{3.6}$$

So the trajectory is always feasible. Trajectory cost is defined with the terminal cost set to Q :

$$\min \sum_{k=0}^{N-1} x_k^T Q x_k + u_k R u_k + x_N Q x_N \tag{3.7}$$

3.1.2 Tuning parameters

The tuning of the Q and R matrices by hand is complicated, as each state and input are in different units and lie in different ranges of values. To make tuning more intuitive, the **Bryson rule** is used. Both Q and R are diagonal matrices containing the **typical values** each state should stay within :

$$Q_{ii} = \frac{1}{x_{i,\text{typ}}^2} \quad R_{ii} = \frac{1}{u_{i,\text{typ}}^2} \tag{3.8}$$

x and y subsystems

For both x and y subsystems, control was tuned for **smooth and slow** behavior, to give time for the roll angle to adjust :

$$\begin{aligned} \omega_{\alpha,\text{typ}} = \omega_{\beta,\text{typ}} &= 10 \ ^\circ/\text{s} \\ \alpha_{\text{typ}} = \beta_{\text{typ}} = \frac{\alpha_{\max}}{2} = \frac{\beta_{\max}}{2} &= 5 \ ^\circ \\ v_{x,\text{typ}} = v_{y,\text{typ}} &= 1 \text{ m/s} \\ \delta_{1,\text{typ}} = \delta_{2,\text{typ}} = \frac{\delta_{1,\max}}{2} = \frac{\delta_{2,\max}}{2} &= 7.5 \ ^\circ \end{aligned} \tag{3.9}$$

z and roll subsystems

For the z and roll subsystems, control was tuned for **fast and reactive** behavior, so the x and y subsystem are adjusting in the right direction :

$$\begin{aligned} v_{z,\text{typ}} &= 1 \text{ m/s} \\ \Delta P_{\text{avg,typ}} &= 10\% \\ \omega_{\gamma,\text{typ}} &= 30 \ ^\circ/\text{s} \\ \gamma_{\text{typ}} &= 1 \ ^\circ \\ P_{\text{diff,typ}} = \frac{P_{\text{diff,max}}}{2} &= 10\% \end{aligned} \tag{3.10}$$

Horizon

The **horizon length** was chosen to be as short as possible while keeping the problem feasible.

$$H = 3 \text{ s} \tag{3.11}$$

3.1.3 Terminal invariant set

Only the x and y subsystems need a terminal controller to guaranty recursive feasibility. Having the same costs, both have terminal sets of the same shape, with reversed velocity axes :

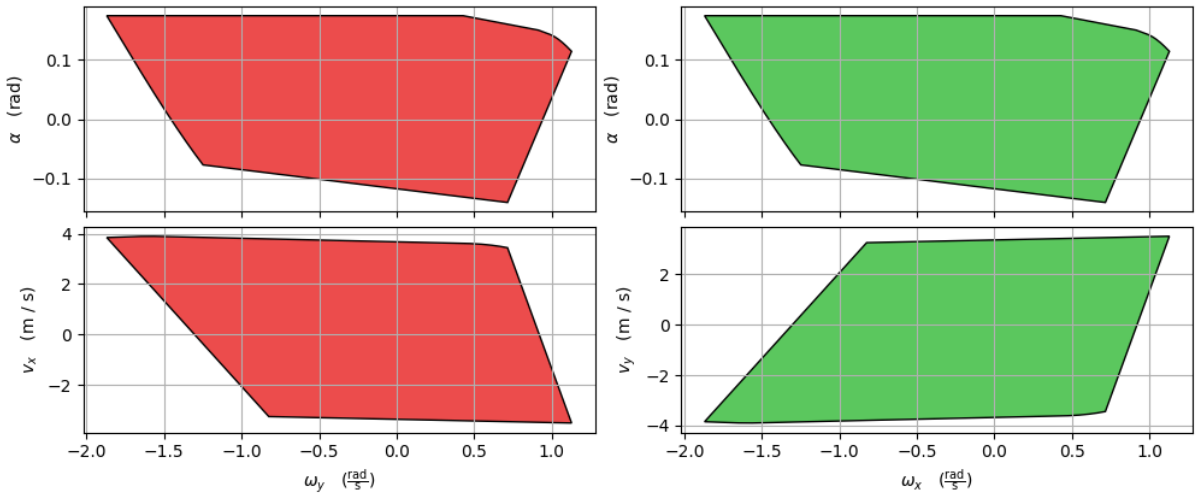

Figure 3.1: x and y subsystems terminal invariant set 2D projections

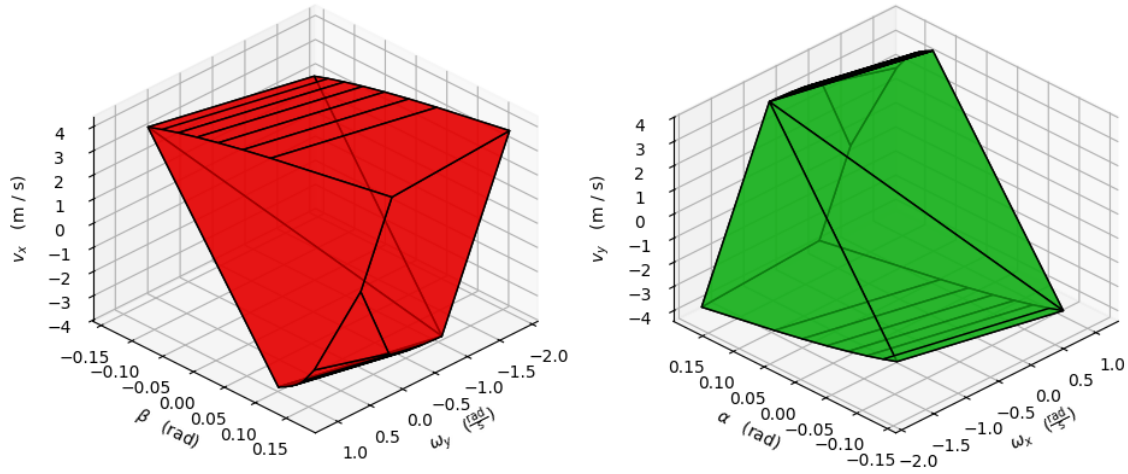

Figure 3.2: x and y subsystems terminal invariant set 3D projection

3.1.4 Plots

Initial drift

Starting from a velocity drift of $(v_x, v_y, v_z) = (5, 5, 5)$ m/s, all subsystems manage to reach steady state within 7 seconds. The z subsystem is a bit faster, reaching steady state in 4 seconds, which is good to avoid losing too much altitude from the start. Because there is no roll error, the roll subsystem stays in steady state.

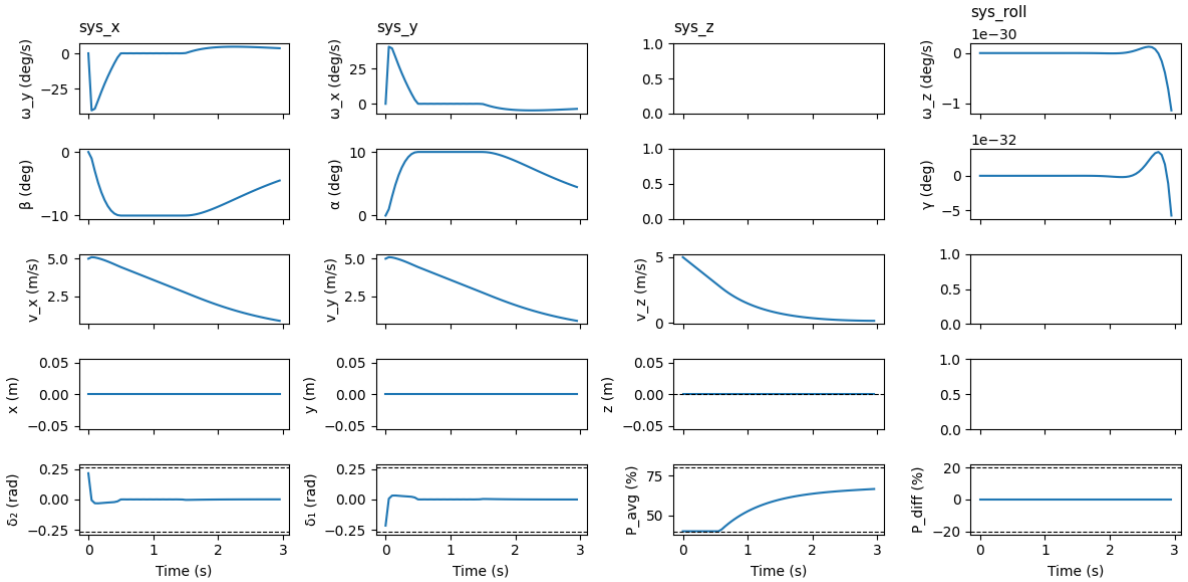

Figure 3.3: 5 m/s in x, y and z stabilization open loop prediction from $t = 0$ s to $t = 3$ s

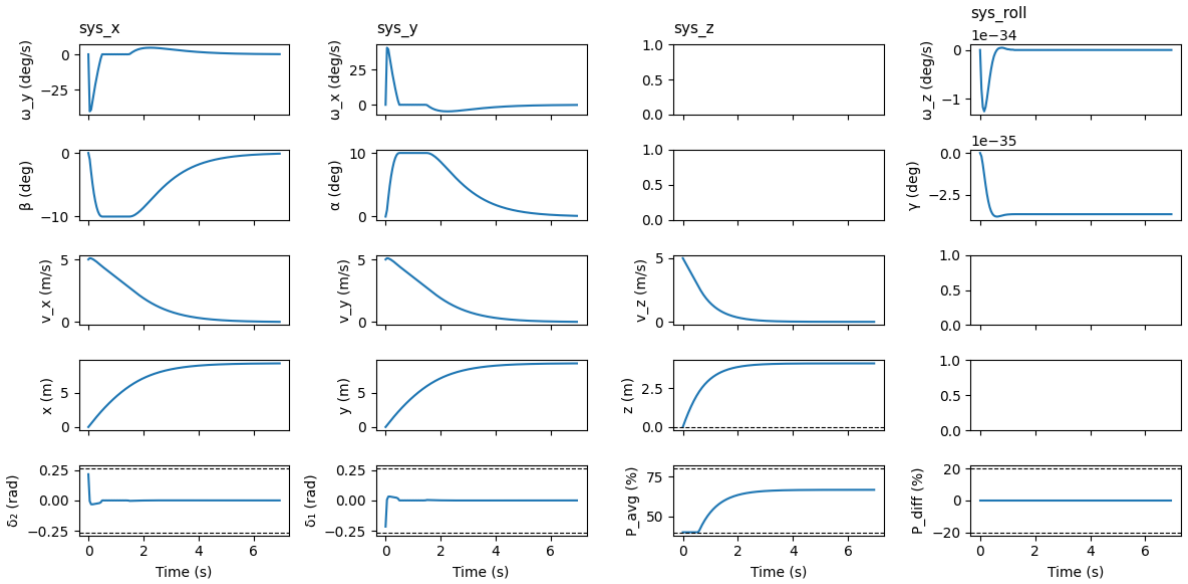

Figure 3.4: 5 m/s in x, y and z stabilization closed loop control from $t = 0$ s to $t = 7$ s

Stationary roll

Starting from a stationary roll error of $\gamma = 30°$, the roll subsystem manages to correct the error within 1.5 seconds. This is desirable, so the x and y subsystems correct for speed in the right direction. Linear positions and velocities as well as (δ_1, δ_2) are in steady state.

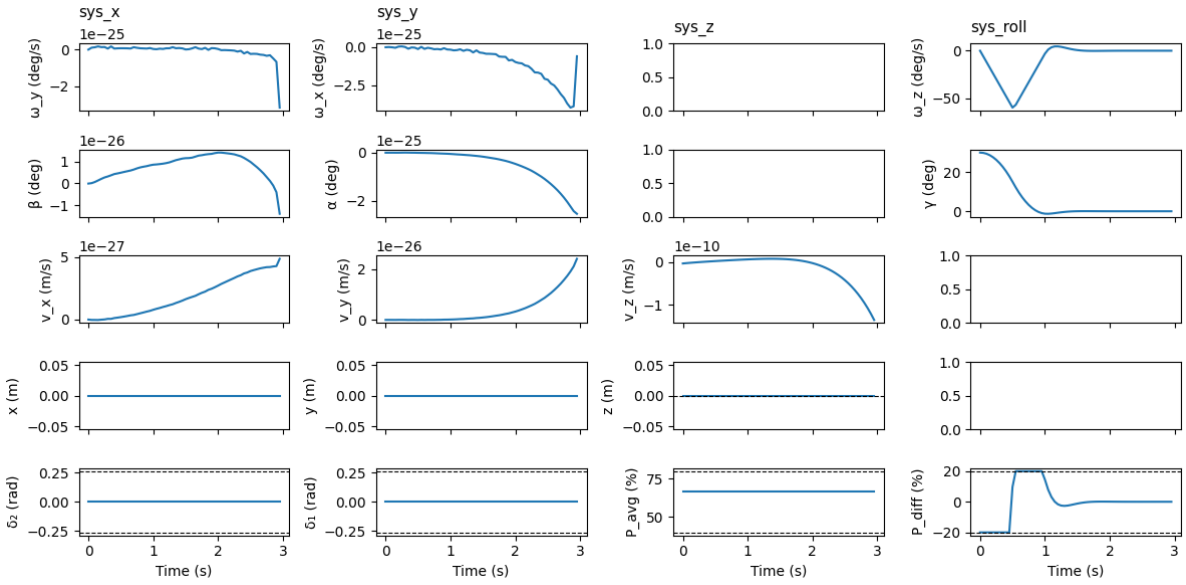

Figure 3.5: Stationary 30° roll stabilization open loop prediction from $t = 0$ s to $t = 3$ s

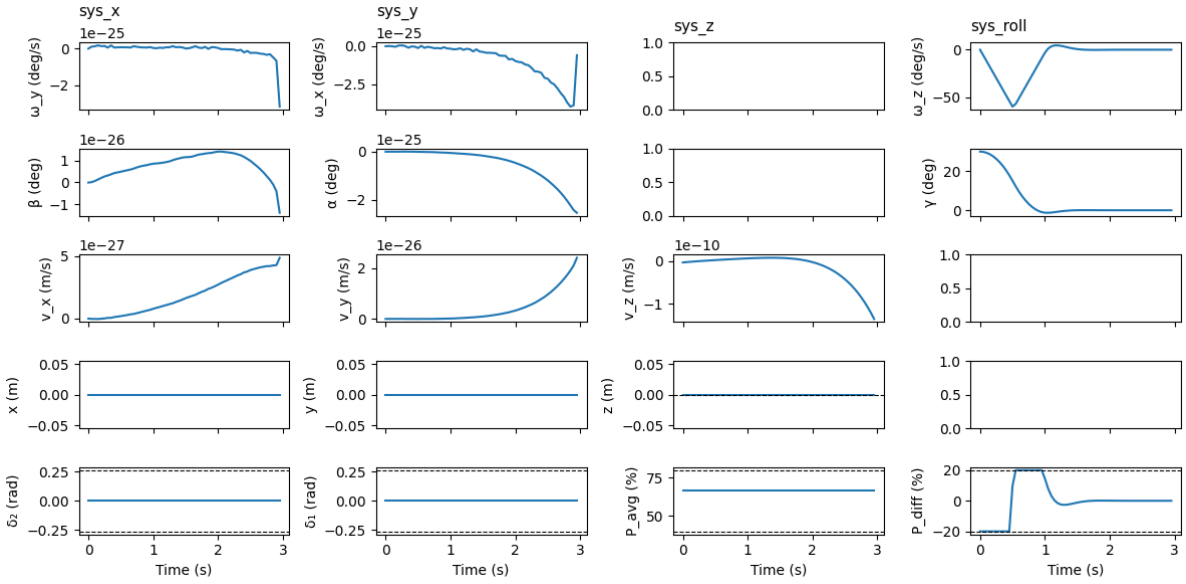

Figure 3.6: Stationary 30° roll stabilization closed loop control from $t = 0$ s to $t = 7$ s

3.2 Velocity tracking

To track a constant reference, the delta formulation is extended with a target state x_t. The rest of the problem stays formulated in the exact same way :

$$\begin{aligned} \Delta x &= x - x_s - x_t \\ \Delta x_0 &= x_0 - x_s - x_t \end{aligned} \tag{3.12}$$

Tuning parameters are still working fine so they are kept **unchanged**. Tracking of a target state $(v_x, v_y, v_z) = (3, 3, 3)$ m/s and $\gamma = 35°$ is reached within 6 seconds. As expected, the roll subsystem reaches steady state first, followed by the z subsystem, and finally the x and y subsystems :

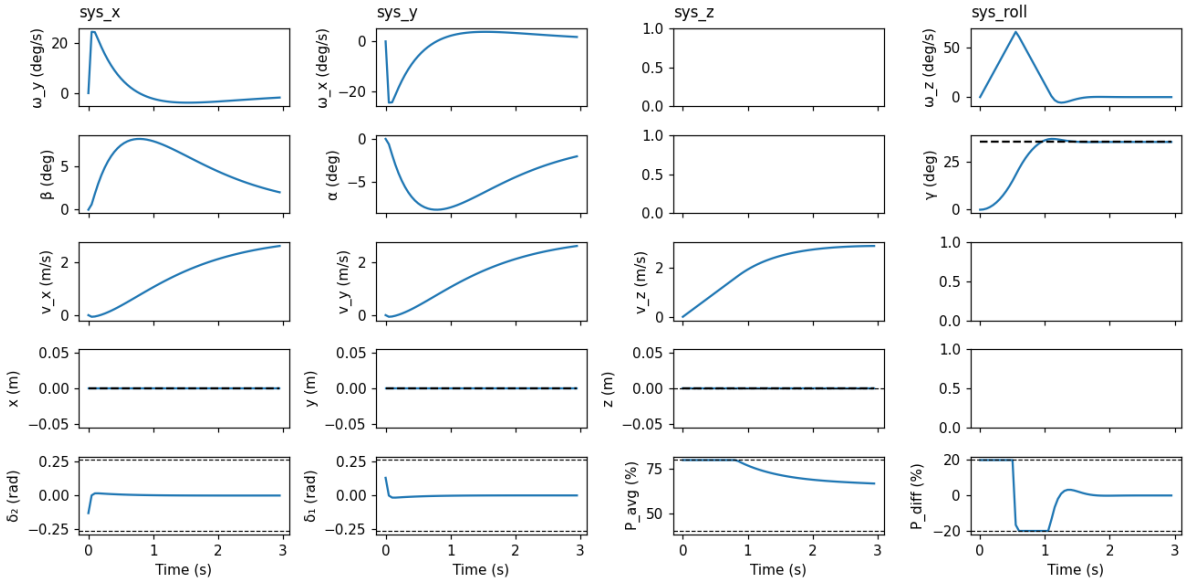

Figure 3.7: Tracking to 3 m/s in x, y, z and 35° roll open loop prediction from $t = 0$ s to $t = 3$ s

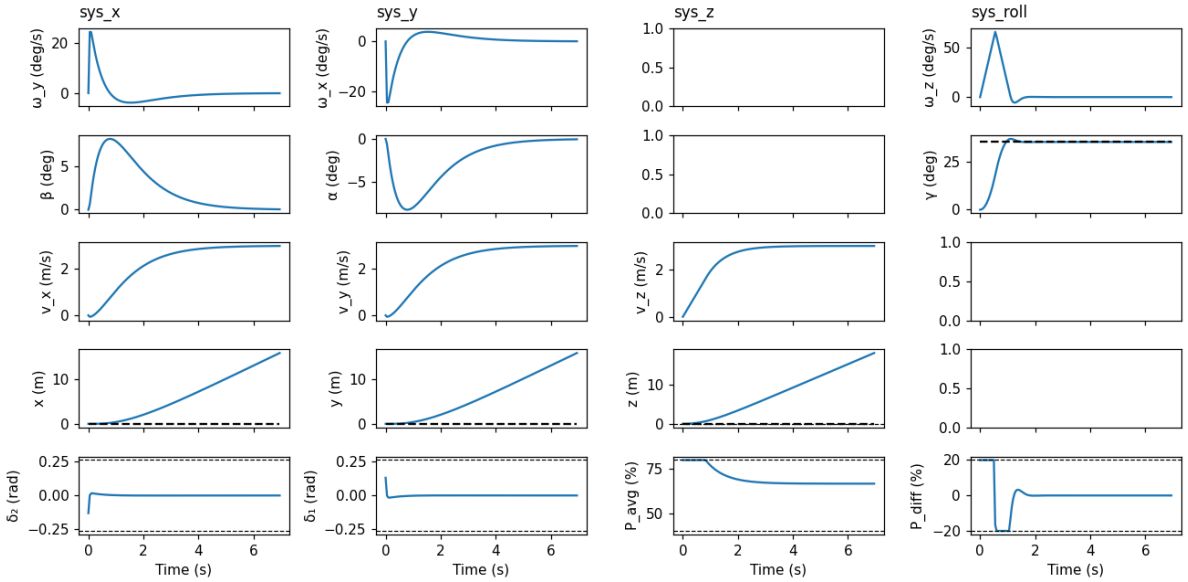

Figure 3.8: Tracking to 3 m/s in x, y, z and 35° roll closed loop control from $t = 0$ s to $t = 7$ s

3.3 Position tracking

Using the given **PID** controller, nothing has to change in the structure of the controller. Here again, the tuning parameters give satisfying results, so they are kept **unchanged**. The controller manage to smoothly reach the position setpoint :

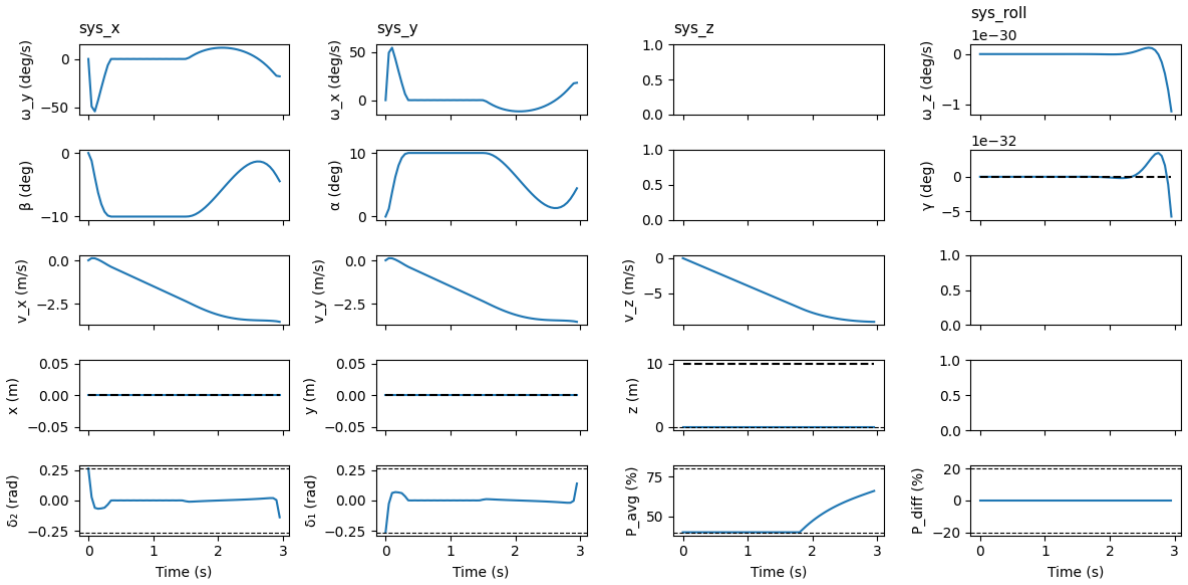

Figure 3.9: Linear landing approach open loop prediction from $t = 0$ s to $t = 3$ s

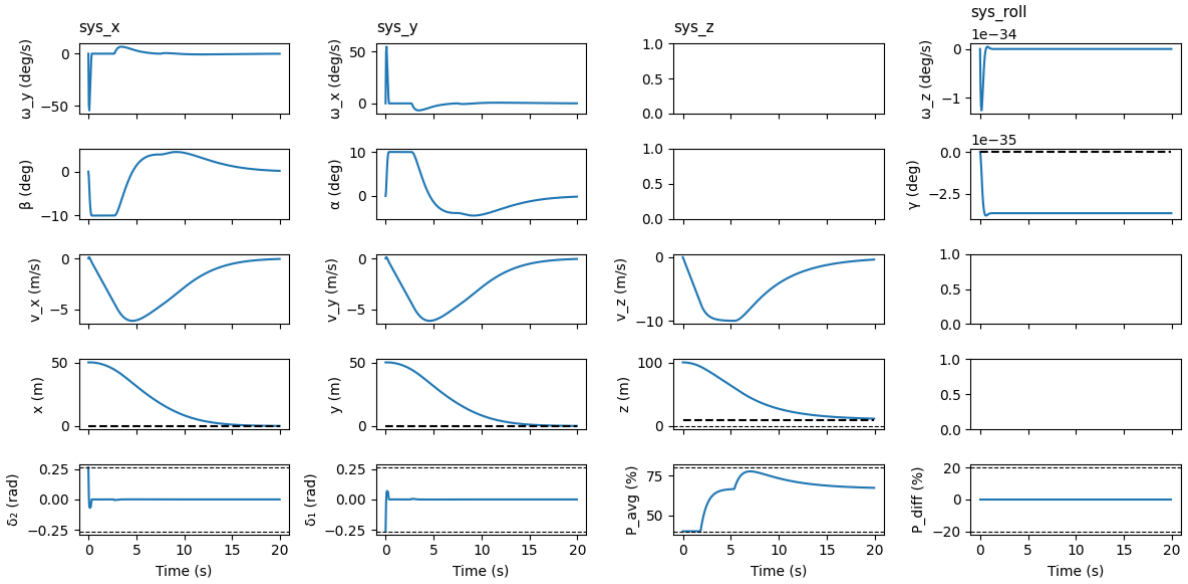

Figure 3.10: Linear landing approach closed loop control from $t = 0$ s to $t = 20$ s

4 Simulation

4.1 Nonlinear simulation

4.1.1 Design procedure

Using the controller from chapter 3 quickly led to constraints violations for both x and y subsystems. To fix this, positive **slack variables** are introduced to soften constraints on angles α and β :

$$\varepsilon \geq 0 \tag{4.1}$$

Hard state constraints and are removed. Because stability cannot be guaranteed anymore, terminal constraints are also removed :

$$\cancel{\begin{matrix} x_k \in \mathcal{X} \quad \forall \quad k \in [0, N-1] \\ x_N \in \mathcal{O}_\infty \end{matrix}} \tag{4.2}$$

These are replaced by soft constraints for both angles :

$$\begin{aligned} |\alpha| &\leq 10^\circ + \varepsilon \\ |\beta| &\leq 10^\circ + \varepsilon \end{aligned} \tag{4.3}$$

The cost is then augmented with those slack variables, and terminal cost is replaced by an additional stage cost Q :

$$\min \sum_{k=0}^{N-1} x_k^T Q x_k + u_k R u_k + S||\varepsilon_k||_1 + x_N Q x_N \tag{4.4}$$

With the slack stage cost S defined in the same way as Q and R, but using the maximum state value and a factor of 10 :

$$S = \frac{10}{\alpha_{\max}^2} = \frac{10}{\beta_{\max}^2} \tag{4.5}$$

With this adapted formulation, constraints are violated for α and β for some time, but the system still stabilizes toward the target position :

4.1.2 Plots

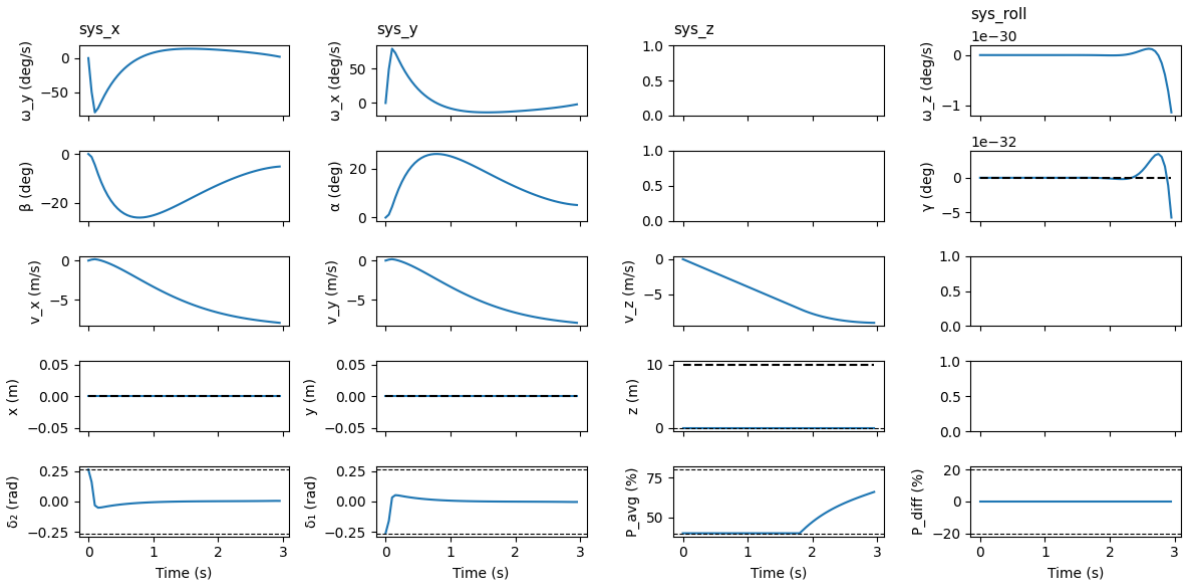

Figure 4.1: Nonlinear landing approach open loop prediction from $t = 0$ s to $t = 3$ s

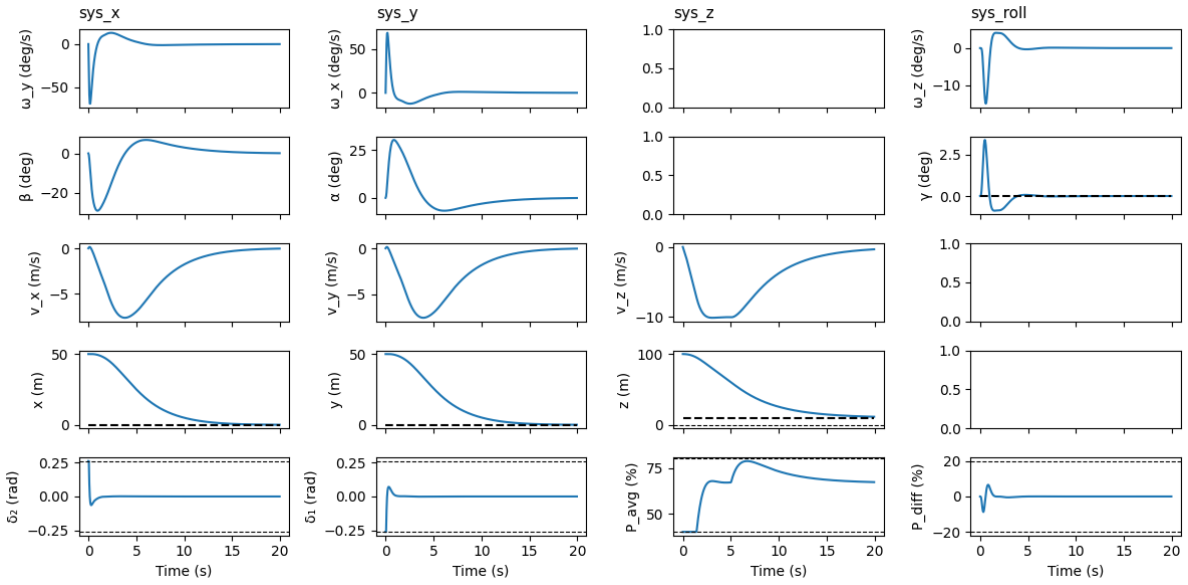

Figure 4.2: Nonlinear landing approach closed loop control from $t = 0$ s to $t = 20$ s

5 Offset Free Tracking

5.1 No change in mass

5.1.1 Design procedure

For **offset-free tracking** on the z subsystem, the dynamics are changed to account for a **constant disturbance** d. We also assume that the state v_z is measurable :

$$\begin{aligned} \Delta x_{k+1} &= A\Delta x_k + B\Delta u_k + B_d d \\ y_k &= Cx_k + C_d d \\ B_d &= B, \quad C = 1, \quad C_d = 0 \end{aligned} \tag{5.1}$$

Augmented states are defined to create an estimator :

$$\hat{A} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \quad \hat{B} = \begin{bmatrix} B \\ 0 \end{bmatrix} \quad \hat{C} = \begin{bmatrix} C & C_d \end{bmatrix} \tag{5.2}$$

Estimator gains are then computed by placing poles P :

$$A - BK = P \Rightarrow L = -K^T \tag{5.3}$$

At the first iteration, the estimated state and disturbance vector is initialized to the first state with zero disturbance :

$$\hat{z}_1 = \begin{bmatrix} \hat{x} \\ \hat{d} \end{bmatrix} = \begin{bmatrix} x_0 \\ 0 \end{bmatrix} \tag{5.4}$$

Then from the second iteration onward, both state and disturbance are estimated using the estimator gains. Estimated state and disturbance $\hat{z}_k$ are then used as the new x_0 and d for the current optimization step. :

$$\begin{aligned} y &= x_0 \\ \hat{z}_k &= \hat{A}\hat{z}_{k-1} + \hat{B}\Delta u_{k-1} + L(\hat{C}\hat{z}_{k-1} - y) \end{aligned} \tag{5.5}$$

First part of the tuning was pole placement. Using reasonable guess $P = \{0.5, 0.7\}$, the disturbance was rejected quickly without oscillations. But the stage cost Q had to be re-tuned for the disturbance to completely disappear :

$$v_{z,\text{typ}} = 0.5 \text{ m/s} \Rightarrow Q = \frac{1}{0.5^2} \tag{5.6}$$

5.1.2 Plots

Using the controller from the previous part, z speed never reaches 0, because the rocket is lighter than its mass at the linearization point. It feels like a constant upward push, making the rocket drift :

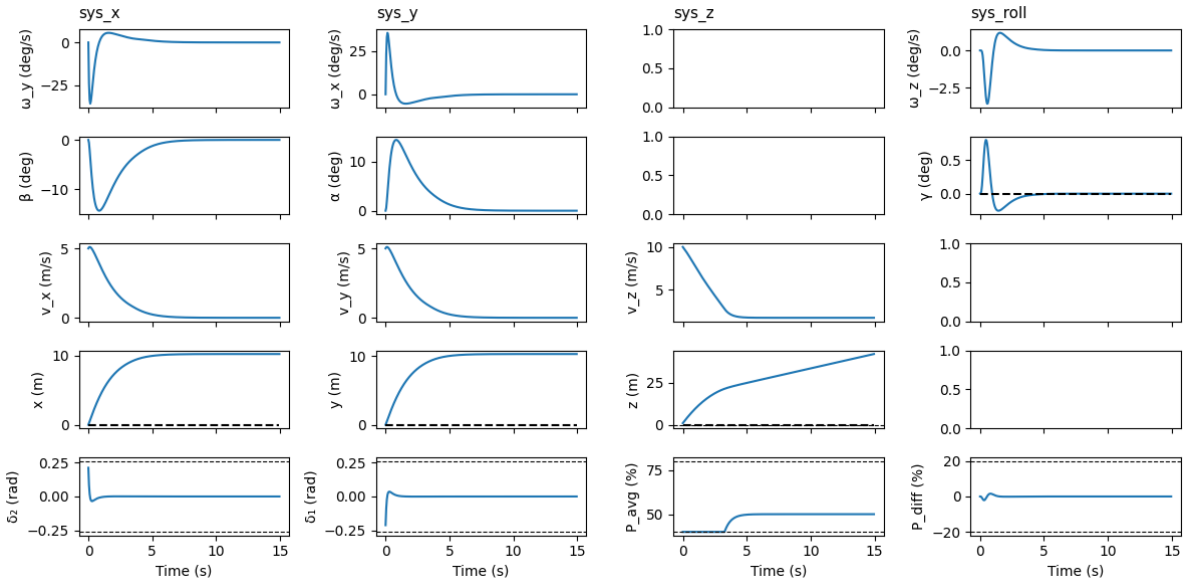

Figure 5.1: Part 4 controller stabilization with constant disturbance

Using the **offset-free** controller from part 5, the disturbance is eventually completely compensated. Because d is constant, once the estimator has converged, the rocket is truly at zero speed, even when zooming on the graph. And the estimator can compensate for any target velocity with any mass, as long as the input range allow it :

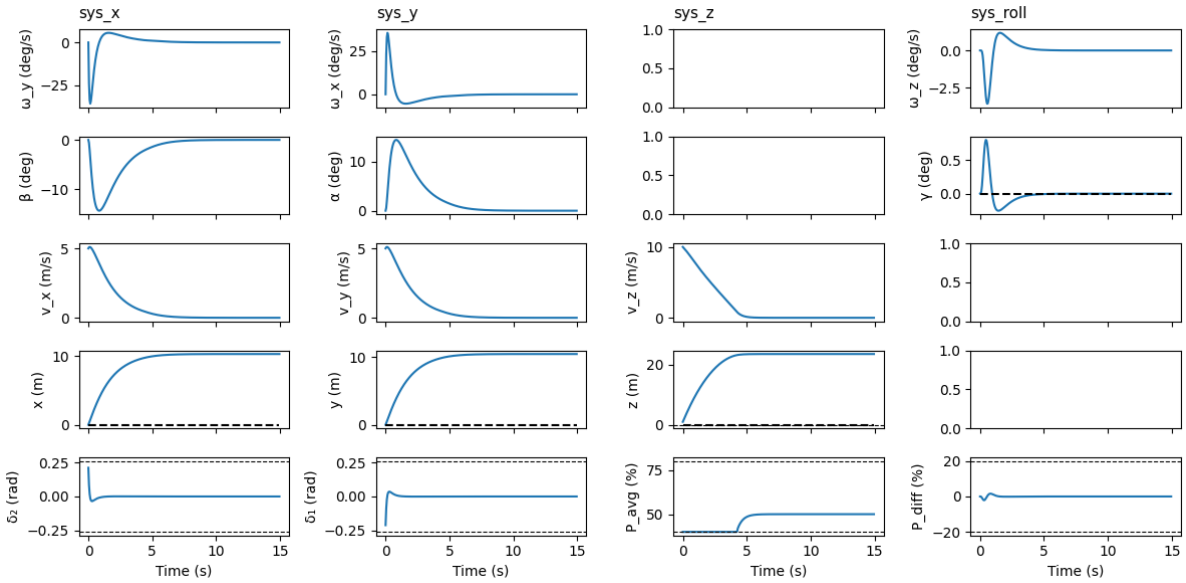

Figure 5.2: Offset-free controller stabilization with constant disturbance

5.2 Change in mass

5.2.1 Plot

For the changing mass case, although it's barely visible on the full scale graph, there is a small velocity offset. And this offset is changing constantly as the rocket loses mass, hence the estimator never catches up.

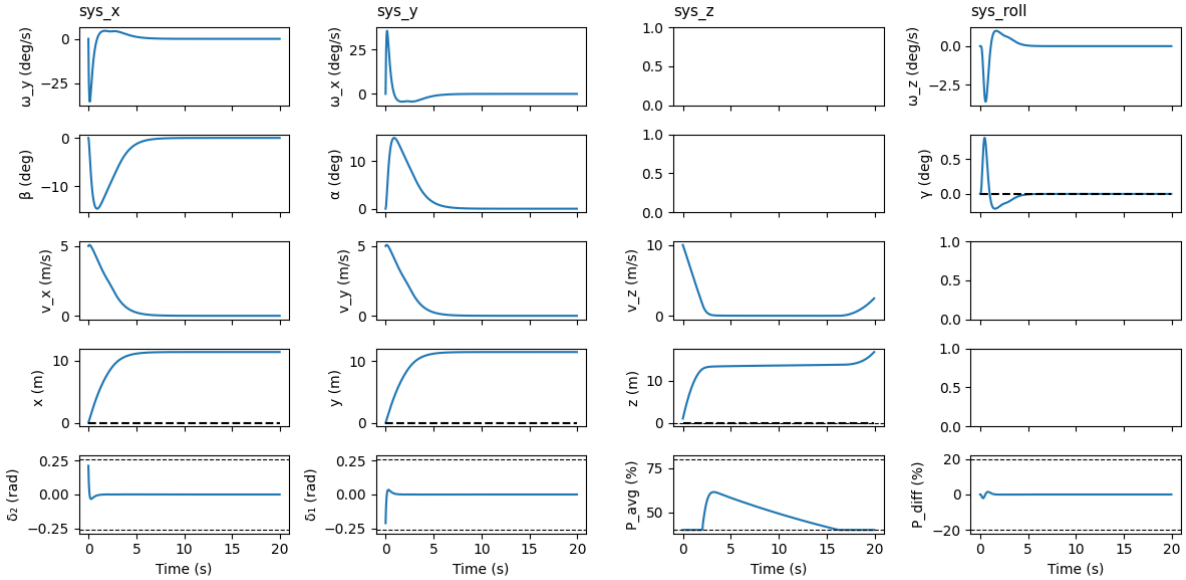

Figure 5.3: Offset-free controller stabilization with variable disturbance

5.2.2 Control stages

Four stages can be observed on this graph :

1. At the start, the controller tries to bleed speed by cutting off the thrust, as it's very far from steady state.
2. After 2 seconds, the controller gets near the target and increases the thrust to slow down near steady state. The disturbance estimate is also converging at the same time.
3. After around 3.5 seconds, the controller has reached a near steady state, but the disturbance keeps changing, so it's always lagging behind trying to decrease thrust.
4. After 16 seconds, the rocket mass goes below what input constraints allow to compensate, so the rocket start accelerating upward until the end of the simulation, occurring at 20 seconds when the rocket is also out of fuel.

5.2.3 Offset correction

We can see from the graph that the change in disturbance is linear. This means that this offset can be eliminated by estimating the disturbance rate of change rather than the disturbance directly with :

$$d_{k+1} = d_k + e \tag{5.7}$$

With e the constant rate of change of the disturbance. This formulation should have integral behavior and correctly compensate input and estimation lag.

6 Tube MPC

6.1 z position controller

6.1.1 Design procedure

For the tube MPC, the z subsystem is modified with a state constraint :

$$z \geq 0 \Rightarrow \mathbb{X} \tag{6.1}$$

And the constant disturbance d is replaced by a bounded disturbance $w \in [-15, 5]$:

$$\boldsymbol{\Delta x}_{k+1} = A\boldsymbol{\Delta x}_k + B\boldsymbol{\Delta u}_k + Bw \tag{6.2}$$

The system is hence split into its nominal delta trajectory and the disturbance rejection controller :

$$\begin{aligned} \boldsymbol{\Delta z}_{k+1} &= A\boldsymbol{\Delta z}_k + B\boldsymbol{\Delta v}_k \\ \boldsymbol{\Delta u}_k &= K(\boldsymbol{\Delta x}_k - \boldsymbol{\Delta z}_k) + \boldsymbol{\Delta v}_k \\ \boldsymbol{\Delta z}_k &\oplus \mathcal{E} \in \mathbb{X} \\ \boldsymbol{\Delta v}_k &\in \mathbb{U} \ominus K\mathcal{E} \end{aligned} \tag{6.3}$$

For the nominal controller, we define tightened state, input and terminal sets :

$$\begin{aligned} \mathcal{X}_f &\subseteq \tilde{\mathbb{X}} = \mathbb{X} \ominus \mathcal{E} \\ K\mathcal{X}_f &\subseteq \tilde{\mathbb{U}} = \mathbb{U} \ominus K\mathcal{E} \\ \mathcal{X}_f &\subseteq \text{pre}(\mathcal{X}_f) \end{aligned} \tag{6.4}$$

With these new constraints, to make the problem feasible the **horizon length has to be extended** to $H = 10$.

Then to have with a **large enough**, **non degenerate** error dynamics, and the stage costs Q and R of the z subsystem are modified with new typical values following the **Bryson rule** :

$$\begin{aligned} \Delta v_{z,\text{typ}} &= 1.3 \text{ m/s} \\ \Delta z_{\text{typ}} &= 0.5 \text{ m} \\ \Delta P_{\text{avg,typ}} &= 17\% \end{aligned} \tag{6.5}$$

6.1.2 Tightened Sets

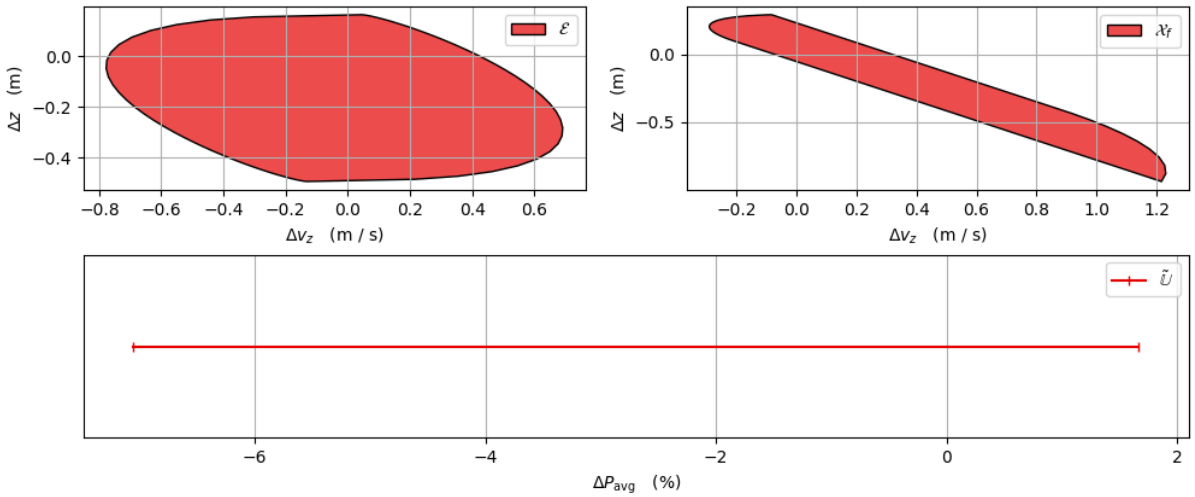

Figure 6.1: Tube MPC error, terminal and input tightened sets

6.1.3 Plots

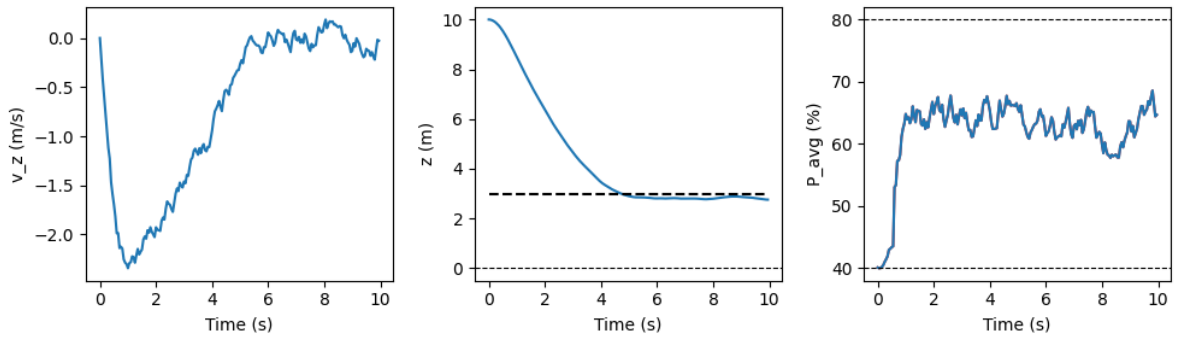

Figure 6.2: z landing system closed loop control under random noise

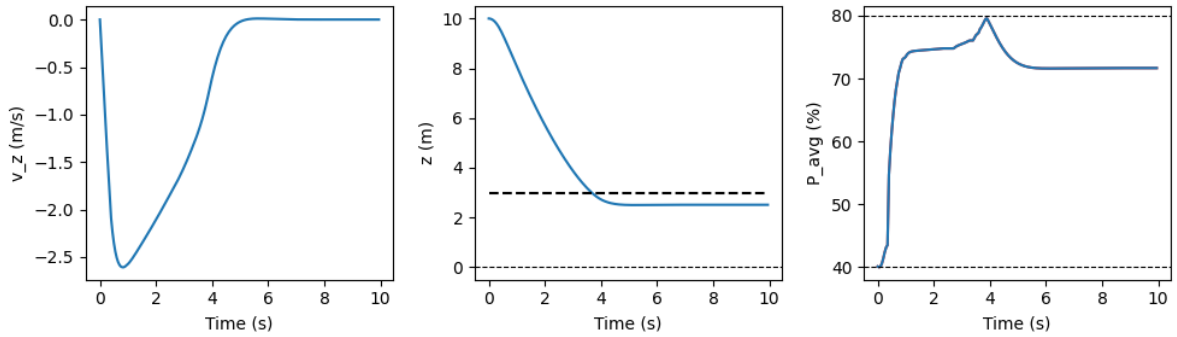

Figure 6.3: z landing system closed loop control under extreme noise

6.2 x, y and roll position controller

6.2.1 Design procedure

Both x and y subsystems are modified to add the position state. Reference tracking is removed to stabilize toward the linearization point which is the target.

$$\begin{aligned} \boldsymbol{\Delta x}_k &= \boldsymbol{x}_k - \boldsymbol{x}_s \\ \boldsymbol{\Delta u}_k &= \boldsymbol{u}_k - \boldsymbol{u}_s \end{aligned} \tag{6.6}$$

In the stage cost Q is extended with a big cost on position error, still using the **Bryson** rule :

$$p_{x,\text{typ}} = p_{y,\text{typ}} = 0.1 \text{ m} \tag{6.7}$$

Then, to avoid breaking soft constraints for too long, the stage cost of angular acceleration is increased, with typical angular acceleration decreased from 10 °/s to 5 °/s, and the soft constraint penalty is increased from 10 to 25 :

$$\begin{aligned} \omega_{\alpha,\text{typ}} = \omega_{\beta,\text{typ}} &= 5 \text{ °/s} \\ S = \frac{25}{\alpha_{\max}^2} &= \frac{25}{\beta_{\max}^2} \end{aligned} \tag{6.8}$$

Finally z and roll subsystems are kept **unchanged**.

6.2.2 Plots

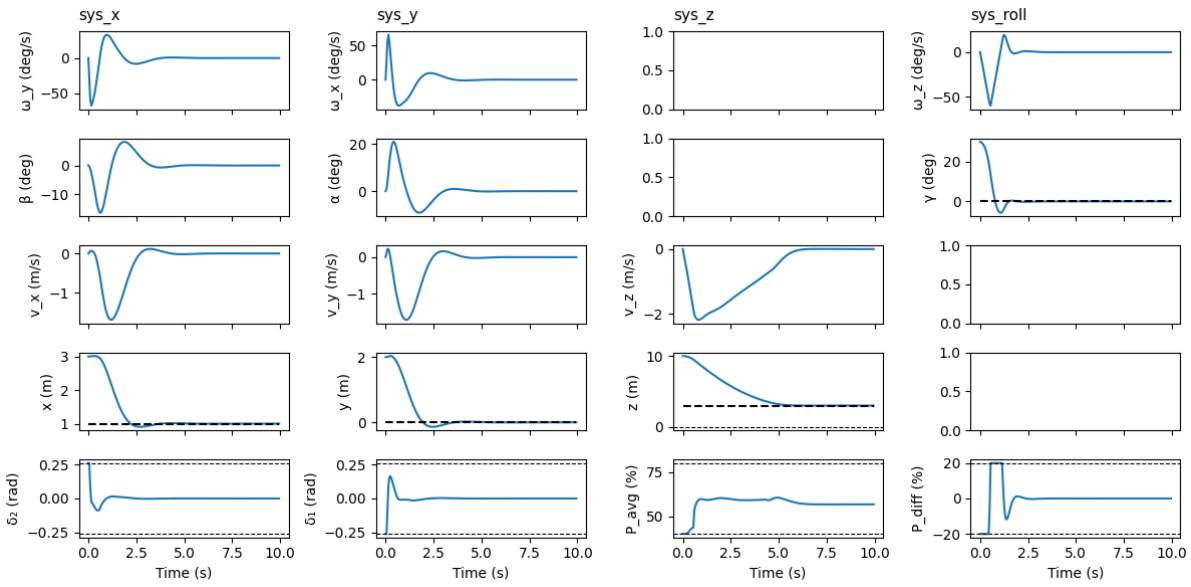

Figure 6.4: Merged controllers closed loop control

7 Nonlinear MPC

7.1 NMPC controller

7.1.1 Design procedure

Using the full nonlinear equations, the system dynamics are :

$$\dot{\boldsymbol{x}} = f(\boldsymbol{x}, \boldsymbol{u}) \tag{7.1}$$

These dynamics are **discretized** using **RK4** :

$$\begin{aligned} k_1 &= h \cdot f\left(\boldsymbol{x}_k, \boldsymbol{u}_k\right) \\ k_2 &= h \cdot f\left(\boldsymbol{x}_k + \frac{k_1}{2}, \boldsymbol{u}_k\right) \\ k_3 &= h \cdot f\left(\boldsymbol{x}_k + \frac{k_2}{2}, \boldsymbol{u}_k\right) \\ k_4 &= h \cdot f\left(\boldsymbol{x}_k + k_3, \boldsymbol{u}_k\right) \\ \boldsymbol{x}_{k+1} &= \boldsymbol{x}_k + \frac{(k_1 + 2 \cdot k_2 + 2 \cdot k_3 + k_4)}{6} \end{aligned} \tag{7.2}$$

The **objective function** is defined using the delta formulation to reach a target $\boldsymbol{x}_t$. The objective is kept as a quadratic cost on state and input, still using the delta formulation around the trim point, for convenience. The costs are still defined with the **Bryson rule** :

$$\omega_{x,\text{typ}} = \omega_{y,\text{typ}} = 30\ ^\circ/\text{s} \quad \omega_{z,\text{typ}} = 60\ ^\circ/\text{s} \qquad \alpha_{\text{typ}} = \beta_{\text{typ}} = 10\ ^\circ \quad \gamma_{\text{typ}} = 1\ ^\circ \tag{7.3}$$

$$v_{x,\text{typ}} = v_{y,\text{typ}} = 1\ \text{m/s} \quad v_{z,\text{typ}} = 2\ \text{m/s} \qquad x_{\text{typ}} = y_{\text{typ}} = z_{\text{typ}} = 0.1\ \text{m} \tag{7.4}$$

$$\delta_{1,\text{typ}} = \delta_{1,\text{typ}} = 10\ ^\circ \qquad \Delta P_{\text{avg,typ}} = 20\ \% \quad \Delta P_{\text{diff,typ}} = 30\ \% \tag{7.5}$$

Then **state constraints** are specified to avoid running into the ground and stay far enough from the Euler angles **gimbal lock** :

$$\begin{aligned} z &> 0 \\ |\beta| &\leq 80^\circ \end{aligned} \tag{7.6}$$

Finally **input constraints** are specified to respect actuators capabilities :

$$\begin{aligned} |\delta_1| &\leq 15^\circ \\ |\delta_2| &\leq 15^\circ \\ 40\% \leq P_{\text{avg}} &\leq 80\% \\ |P_{\text{diff}}| &\leq 20\% \end{aligned} \tag{7.7}$$

Also, the horizon length is set to $H = 6$, to try to match the horizon length of the **tube MPC** of part 6, while keeping the computation time reasonable.

7.1.2 Plots

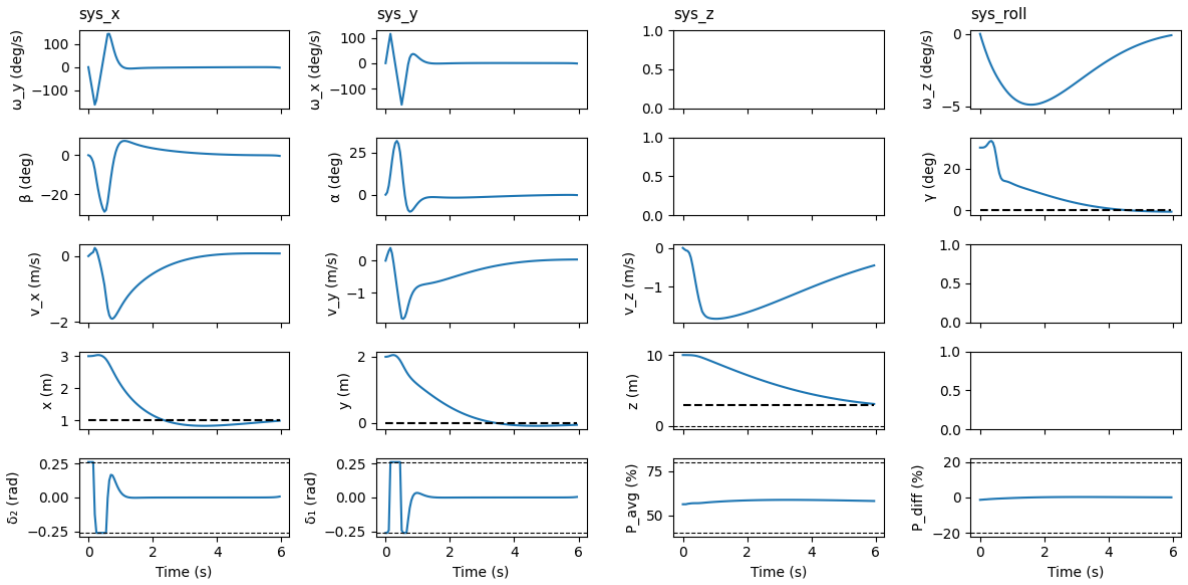

Figure 7.1: Open loop prediction at $t = 0$ s

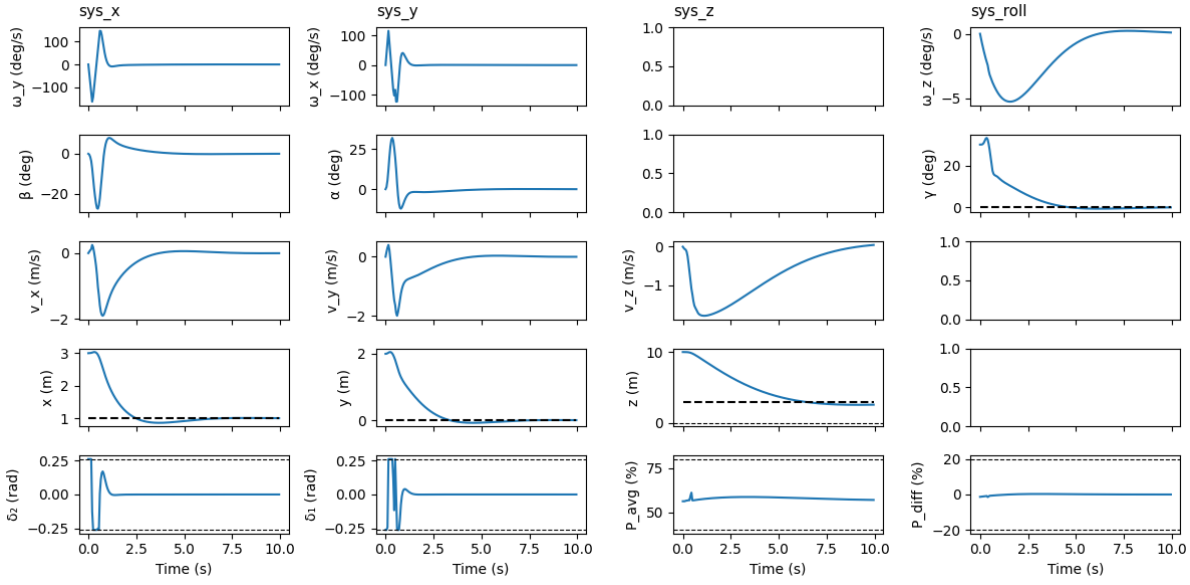

Figure 7.2: Closed loop control from $t = 0$ s to $t = 4$ s

7.1.3 Comparison

Even though the nonlinear formulation makes it easier to formulate the problem with a greater tuning range, it makes it harder to choose a longer horizon which could dampen oscillations. Otherwise, it seems reasonable to think that the nonlinear MPC could achieve much better results than the **tube MPC** formulation.